

Agentic-OS-V2

Dario Anchundia

Las entidades representadas en el proyecto son: las ventanas gráficas (`x11_client`), el proceso **Launcher** y el servidor **IA Learner**. Debido a que la estación de trabajo del usuario y el servidor de clasificación son entidades físicamente separadas, se optó por sockets TCP para el intercambio de datos entre ambos, reservando las señales POSIX (`SIGCHLD`) exclusivamente para el control local entre el **Launcher** y sus procesos hijos. Cada tecla presionada en una ventana se envía carácter por carácter al servidor, donde los hilos de conexión actúan como productores que agrupan las letras en oraciones al detectar un retorno de línea (`RET`) y las depositan de forma concurrente en una cola global de trabajo. Un hilo orquestador (**Loader**) administra el procesamiento por lotes, esperando hasta reunir P oraciones para reactivar simultáneamente a un grupo de P hilos detectores (consumidores). Estos hilos balancean su ejecución distribuyéndose en los núcleos de CPU disponibles mediante afinidad del procesador, calculan las frecuencias de términos mediante Bag of Words y, apenas se clasifica el documento editado en una ventana, infieren el contexto de usuario de manera asincrónica.

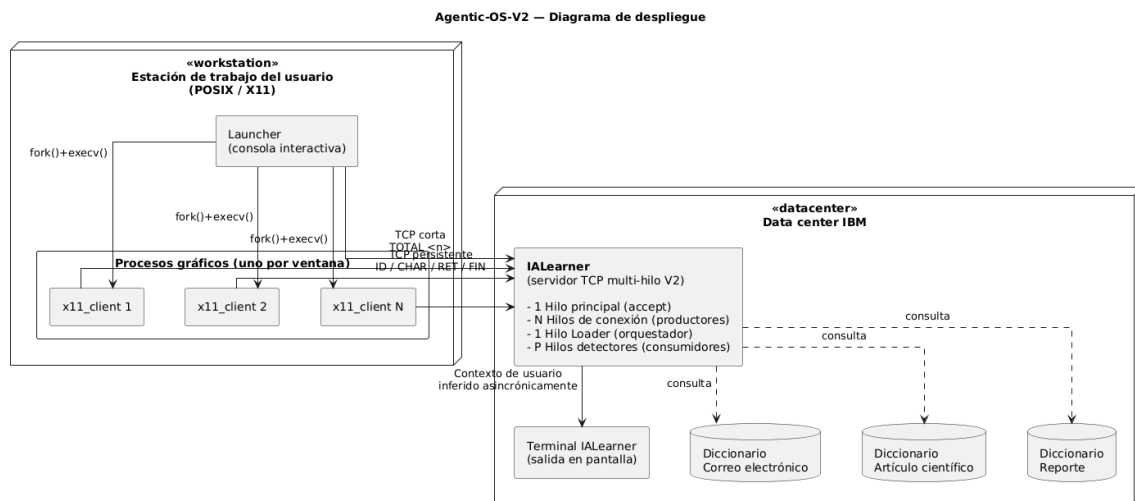


Figura 1: Diagrama de despliegue de Agentic-OS-V2

Componentes

Launcher (proceso, consola interactiva):

1. Valida los parámetros de entrada (N , host, puerto).
2. Notifica al **IA Learner**, mediante una conexión TCP corta, el mensaje `TOTAL <n>` con el número de ventanas que debe esperar.
3. Crea cada proceso gráfico (`x11_client`) mediante las llamadas al sistema `fork()` y `execv()`.
4. Despliega un menú interactivo para monitorear o cerrar los procesos y define el manejador de señal `SIGCHLD` con `waitpid(-1, &estado, WNOHANG)` en un bucle `while`, garantizando cero procesos zombie sin bloquear la consola.

x11_client (un proceso por ventana):

1. Se conecta por TCP al **IA Learner** y se identifica mediante `ID <n>`.
2. Envía cada tecla imprimible mediante `CHAR <c>` en ráfagas asíncronas sin bloquear el hilo de dibujado gráfico.
3. Al presionar Enter envía `RET` (fin de oración); al presionar Escape envía `FIN` y cierra la ventana.

IA Learner (servidor TCP multi-hilo V2):

1. Acepta múltiples conexiones y atiende cada una en un hilo independiente (`pthread_create`), los cuales actúan como productores acumulando caracteres y encolando oraciones completas en una estructura protegida por exclusión mutua.
2. Un hilo orquestador (**Loader**) permanece suspendido en espera pasiva mediante `pthread_cond_wait` hasta que se hayan acumulado al menos P oraciones pendientes en la cola.
3. Al completarse el lote, el **Loader** despierta de forma simultánea (`pthread_cond_broadcast`) a un grupo de P hilos detectores en paralelo. Si llegan más de P oraciones, los hilos en exceso esperan hasta la siguiente activación.
4. Cada hilo detector fija su afinidad de hardware (`pthread_setaffinity_np`) para distribuir y balancear la carga uniformemente sobre las CPUs disponibles.
5. Apenas se procesa una oración y se clasifica el documento de una ventana, el hilo detector evalúa inmediatamente y de forma asincrónica la matriz de decisión para determinar e imprimir el tipo de usuario.

6. Toda impresión en consola está serializada mediante un mutex global (`g_mutex_consola`) para evitar el entrelazado de caracteres.

Protocolo interno (IPC vía sockets TCP)

Cada línea enviada por socket termina en salto de línea. Mensajes definidos en `protocolo.h`:

Mensaje	Dirección	Significado
ID <n>	Ventana → IALearner	La ventana <i>n</i> se identifica al conectar
CHAR <c>	Ventana → IALearner	Se presionó la tecla imprimible <i>c</i>
RET	Ventana → IALearner	Fin de oración (Enter), dispara el encolado en el servidor
FIN	Ventana → IALearner	La ventana se cerró (Escape)
TOTAL <n>	Launcher → IALearner	Notificación de ventanas creadas en la ronda

Clasificación (Bag of Words) y Detección Asíncrona

Un documento se clasifica en una categoría si aparecen al menos 3 coincidencias de su respectivo diccionario. Si califica para más de una clase, se asigna la de mayor frecuencia acumulada.

Correo electrónico	Artículo científico	Reporte
thank, please, regards, meeting, attached, information, update, schedule, team, project	data, analysis, results, method, study, model, research, system, significant, effect	system, data, network, security, application, server, user, performance, service, infrastructure

Apenas un hilo detector determina la clase de un documento, evalúa de manera asíncrona la tabla de perfiles (X = presencia binaria de al menos un documento con esa etiqueta):

Tipo de usuario	Correo	Artículo	Reporte
Personal administrativo	X		X
Personal técnico	X		
Profesor	X	X	
Estudiante		X	X

Concurrencia, Afinidad y Programación Defensiva

El servidor implementa un patrón Productor-Consumidor por lotes. Los hilos de red insertan oraciones en una cola compartida protegida por exclusión mutua, mientras que el hilo **Loader** y los P hilos detectores sincronizan la ejecución paralela mediante variables de condición (`pthread_cond_wait` y `pthread_cond_broadcast`), erradicando la espera activa y manteniendo el consumo de CPU en 0% durante los periodos de inactividad. Para optimizar el rendimiento multi-núcleo, se distribuyen los hilos detectores balanceando la carga en los procesadores del sistema mediante `pthread_setaffinity_np`. En el **Launcher**, el manejador de `SIGCHLD` recolecta de forma no bloqueante (`WNOHANG`) los recursos de los procesos terminados, evitando procesos zombie sin pausar la consola interactiva. Finalmente, toda emisión a la terminal está serializada bajo un mutex global.

Compilación y Ejecución

```
gcc ia_learner/ia_learner.c -o ia_learner/ia_learner -Wall -Wextra -pthread -D_GNU_SOURCE
gcc launcher/launcher.c -o launcher/launcher -Wall -Wextra
gcc x11_client/x11_client.c -o x11_client/x11_client -Wall -Wextra -lX11
```

Terminal 1 (servidor): `./ia_learner/ia_learner <puerto><P>` (ejemplo: `./ia_learner/ia_learner 9500 4`).

Terminal 2 (launcher): `./launcher/launcher <N_ventanas>[host] [puerto]`.